

BRANDNEST

Manage every brand, from one place

AI CODING SPEC | Version 4.0 | 2026
Feed this document to your AI coding assistant and start building

Field	Value
Product	BrandNest — multi-brand editorial management platform
User Model	Single owner. No teams, no roles, no public sign-up.
Base Starter	MakerKit Next.js Supabase SaaS Kit Lite (Turborepo + pnpm)
Framework	Next.js 15 App Router — TypeScript only
Database	Supabase (PostgreSQL + GoTrue Auth + Vault + Storage)
Hosting	Vercel (auto-deploy from GitHub)
AI Proxy	Cloudflare AI Gateway → OpenRouter
IDE	Google Antigravity IDE + Google Stitch MCP
Repo	github.com/YOUR-USERNAME/brandnest
Document Type	Vibe-coding spec — no timelines, no Gantt, no traditional PM

1. What Is BrandNest

A personal command centre for every brand you own

BrandNest is a single-owner multi-brand platform. One account. Multiple brands. Each brand has its own WordPress site, Google Analytics property, social media channels, and AI-generated content library. The app lets you switch between brands instantly, with full context isolation at the database level.

The app has two scopes that share the same codebase:

MVP Scope (Build First)

Brand configuration hub: create brands, store WordPress credentials (via Supabase Vault), set social channel URLs, sitemap URLs, and N8N/automation webhook endpoints. Instantly useful to any N8N or Postiz workflow. Every external automation tool reads per-brand config from /api/mcp.

Full Product Scope (Build After)

AI Content Studio (6-step pipeline: research → outline → draft → SEO → social → publish), social media calendar, analytics dashboard (GA4 + GSC + AdSense), WordPress post sync, and brand asset library.

1.1 Core Architectural Constraints

These are not preferences — they are hard constraints the AI must respect in every file it generates:

Constraint	Rule
Single Owner	Every table has owner_id UUID referencing auth.users(id). NO brand_members table. NO roles column. NO invitation system.
RLS at DB Layer	Every table has a Supabase RLS policy: USING (owner_id = auth.uid()) WITH CHECK (owner_id = auth.uid()). App code also filters by owner_id for defence in depth.
Brand Slug Routing	Active brand is in the URL: /home/brands/[brandSlug]. BrandContext reads from URL param — not from React state alone. Slugs are auto-generated on create and NEVER changed.
Vault for Secrets	WordPress app passwords and sensitive OAuth tokens go into Supabase Vault via service role key. Tables store the vault key reference, not the plain text secret. Never return vault values to the browser client.
No Server	Next.js App Router + Server Components + Route Handlers only. No NestJS. No Express. No Docker. Serverless only.
No Public Sign-up	Disable sign-up in Supabase Auth settings. The owner creates the account once and that is the only account that ever exists.

2. Full Tech Stack

Exact packages — use these, nothing else without approval

Layer	Package / Service	Notes
Starter	MakerKit Next.js Supabase SaaS Kit Lite	Clone this — do NOT start from scratch or use vercel/nextjs-supabase-starter
Monorepo	Turborepo + pnpm workspaces	apps/web (Next.js), packages/ui, packages/supabase
Framework	Next.js 15 App Router (TypeScript)	App Router only. No Pages Router.
UI	TailwindCSS + shadcn/ui	Dark mode native via CSS variables. Do not add a separate dark mode library.
Auth	Supabase GoTrue	Email/password + magic link. Provided by MakerKit.
Database	Supabase PostgreSQL	All data. RLS on every table.
Secret Storage	Supabase Vault	WordPress app passwords + OAuth tokens only.
File Storage	Supabase Storage	Bucket: brand-assets. Path: /{owner_id}/{brand_id}/logo.png
Background Jobs	Trigger.dev	Social post scheduler. Every-minute cron job.
AI Proxy	Cloudflare AI Gateway → OpenRouter	All AI calls route through Cloudflare AI Gateway URL.
MCP Hub	physics91/openrouter-mcp	Single MCP server. GA4 + GSC + WordPress + Supabase registered as tools.
Deep Research MCP	wheattoast11/openrouter-deepresearch-mcp	Used for Step 1 competitor research only.
App MCP Server	@vercel/mcp-adapter at /api/mcp	Exposes per-brand config to external tools (N8N, Postiz).
Rich Text	TipTap	Article drafting + social post composer.
Drag and Drop	@dnd-kit/core	Kanban pipeline + outline builder.
Calendar	@fullcalendar/react	Social scheduling calendar view.
Charts	Recharts	Analytics dashboards.
Analytics Cache	analytics_cache table (PostgreSQL)	6-hour TTL cache for GA4 + GSC responses.

2.1 AI Model Routing

All AI calls go through lib/ai-router.ts. Route to the cheapest model that produces acceptable output.

Task	Model	Cost Tier
Article research + competitor analysis	Perplexity Sonar Pro (via OpenRouter)	Paid — web search
Long-form article drafting (Step 3)	Claude Sonnet 4.6 (claude-sonnet-4-6)	Paid — best quality
Social post repurposing (Step 5)	Claude Haiku 4.5 (claude-haiku-4-5-20251001)	Low cost
Outline + internal links + duplicate check	Gemini 2.0 Flash (google/gemini-flash-2.0)	Low cost
Titles / meta description / hashtags / alt text	Gemma 4 31B Instruct (google/gemma-4-31b-it:free)	FREE
Featured image generation	FLUX.2 Max (black-forest-labs/flux-2-max)	Paid — image
Content idea generation	Gemma 4 31B Instruct (google/gemma-4-31b-it:free)	FREE

```
// lib/ai-router.ts – all AI calls go through this
const GATEWAY = process.env.CLOUDFLARE_AI_GATEWAY_URL; // ends with /openrouter

export const AI_MODELS = {
  research: 'perplexity/sonar-pro',
  write:    'anthropic/claude-sonnet-4-6',
  social:   'anthropic/claude-haiku-4-5-20251001',
  outline:  'google/gemini-flash-2.0',
  free:     'google/gemma-4-31b-it:free',
  image:    'black-forest-labs/flux-2-max',
} as const;

export async function callAI(model: string, messages: any[], opts = {}) {
  const res = await fetch(`${GATEWAY}/chat/completions`, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.OPENROUTER_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ model, messages, ...opts }),
  });
  return res.json();
}
```

3. Monorepo File Structure

Every file and folder the AI needs to know about

```
brandnest/                                # Turborepo monorepo root (from MakerKit Lite)
├── apps/
│   ├── web/                              # Next.js 15 App Router application
│   │   ├── app/
│   │   │   ├── (marketing)/              # Public landing page – MakerKit marketing layout
│   │   │   │   ├── page.tsx              # Landing page with hero, features, CTA
│   │   │   │   ├── auth/                # Auth pages provided by MakerKit
│   │   │   │   │   ├── login/page.tsx
│   │   │   │   │   └── callback/route.ts
│   │   │   │   ├── home/                # Protected app shell
│   │   │   │   │   ├── layout.tsx        # Sidebar + BrandContext provider
│   │   │   │   │   └── page.tsx          # Brand picker / overview dashboard
│   │   │   │   └── brands/
│   │   │   │       └── [brandSlug]/
│   │   │   │           ├── layout.tsx     # Per-brand shell, sets BrandContext from slug
│   │   │   │           ├── page.tsx       # Brand dashboard
│   │   │   │           └── content/
│   │   │   │               ├── page.tsx   # Content library (all articles for this brand)
│   │   │   │               ├── new/page.tsx # AI Content Studio (6-step wizard)
│   │   │   │               └── [articleId]/page.tsx # Article editor
│   │   │   │               └── social/
│   │   │   │                   ├── page.tsx # Social media calendar
│   │   │   │                   ├── compose/page.tsx # Social post composer
│   │   │   │                   ├── analytics/page.tsx # GA4 + GSC + AdSense dashboard
│   │   │   │                   └── settings/page.tsx # Brand settings
│   │   └── api/
│   │       ├── mcp/route.ts              # App MCP server via @vercel/mcp-adapter
│   │       └── ai/
│   │           ├── research/route.ts
│   │           ├── outline/route.ts
│   │           ├── draft/route.ts
│   │           ├── seo/route.ts
│   │           ├── social/route.ts
│   │           └── publish/route.ts        # Push to WordPress via wp-json REST
│   └── components/
│       ├── brand/
│       │   ├── BrandCard.tsx
│       │   ├── BrandSwitcher.tsx
│       │   └── CreateBrandModal.tsx
│       ├── content/
│       │   ├── ContentStudio.tsx          # 6-step wizard wrapper
│       │   ├── StepResearch.tsx
│       │   ├── StepOutline.tsx
│       │   ├── StepDraft.tsx
│       │   ├── StepSEO.tsx
│       │   ├── StepSocial.tsx
│       │   └── StepPublish.tsx
│       ├── social/
│       │   ├── SocialCalendar.tsx
│       │   └── PostComposer.tsx
│       ├── analytics/
│       │   ├── AnalyticsDashboard.tsx
│       │   └── StatCard.tsx
│       └── lib/
│           ├── ai-router.ts               # All AI calls – model selection + Cloudflare gateway
│           ├── vault.ts                   # Supabase Vault read/write (service role only)
│           ├── wordpress.ts               # wp-json REST client (publish, sync posts)
│           └── analytics.ts                # GA4 + GSC fetch helpers with 6h cache
```

```
■      ■■■ context/
■      ■ ■■■ BrandContext.tsx # Active brand provider – reads from URL slug
■      ■■■ supabase/
■      ■■■ migrations/      # All DB migrations go here – committed to git
■■■ packages/
■   ■■■ ui/                  # Shared shadcn/ui components
■   ■■■ supabase/            # Shared Supabase client helpers
■■■ trigger.config.ts        # Trigger.dev jobs
■■■ turbo.json
■■■ pnpm-workspace.yaml
```

4. Database Schema

Full SQL — apply via Supabase migrations

Migration Rule

Every schema change goes in `apps/web/supabase/migrations/` as a timestamped `.sql` file. Run: `pnpm run supabase:web:reset` to apply locally. Never edit the schema directly in the Supabase dashboard for anything you want to survive a reset.

4.1 brands

```
create table public.brands (
  id            uuid primary key default gen_random_uuid(),
  owner_id      uuid not null references auth.users(id) on delete cascade,
  name          text not null,
  slug          text not null unique, -- auto-generated, immutable
  tagline       text,
  description    text,
  niche         text,
  logo_url      text,                -- Supabase Storage URL
  primary_color text default '#1A56DB',
  -- WordPress
  wordpress_url      text,
  wordpress_username text,
  wordpress_vault_key text,          -- Supabase Vault key for app password
  -- Sitemap
  sitemap_url text,
  -- Social channel profile URLs
  instagram_url text, twitter_url text, facebook_url text,
  pinterest_url text, youtube_url text, tiktok_url text,
  linkedin_url text, medium_url text, reddit_url text,
  threads_url text,
  -- Automation webhooks
  n8n_webhook_url text,
  postiz_webhook_url text,
  mixpost_webhook_url text,
  extra_integrations jsonb default '{}',
  -- Analytics
  ga4_property_id text,
  gsc_property_url text,
  adsense_publisher_id text,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

alter table public.brands enable row level security;
create policy owner_only on public.brands
  using (owner_id = auth.uid())
  with check (owner_id = auth.uid());

-- Slug generation trigger
create or replace function generate_brand_slug()
returns trigger language plpgsql as $$
begin
  if NEW.slug is null or NEW.slug = '' then
    NEW.slug := lower(regexp_replace(NEW.name, '^[a-zA-Z0-9]+', '-', 'g'))
              || '-' || substr(gen_random_uuid()::text, 1, 6);
  end if;
  return NEW;
end;
```

```
end; $$;
```

```
create trigger set_brand_slug before insert on public.brands
  for each row execute function generate_brand_slug();
```

4.2 articles

```
create table public.articles (
  id          uuid primary key default gen_random_uuid(),
  owner_id    uuid not null references auth.users(id) on delete cascade,
  brand_id    uuid not null references public.brands(id) on delete cascade,
  -- Content
  title       text not null,
  slug        text,
  content      text,          -- TipTap JSON or HTML
  excerpt     text,
  featured_image text,        -- Supabase Storage URL
  -- SEO
  meta_title   text,
  meta_description text,
  focus_keyword text,
  alt_text     text,
  -- AI Studio state
  studio_step  int default 0,  -- 0-5: which step of the 6-step wizard
  research_data jsonb default '{}',
  outline_data jsonb default '{}',
  seo_data     jsonb default '{}',
  social_data  jsonb default '{}',
  -- Publishing
  status       text default 'draft', -- draft | ready | published
  wp_post_id   int,
  wp_post_url  text,
  published_at timestamptz,
  -- Duplicates
  duplicate_score float,
  created_at   timestamptz default now(),
  updated_at   timestamptz default now()
);

alter table public.articles enable row level security;
create policy owner_only on public.articles
  using (owner_id = auth.uid())
  with check (owner_id = auth.uid());
```

4.3 social_accounts

```
create table public.social_accounts (
  id          uuid primary key default gen_random_uuid(),
  owner_id    uuid not null references auth.users(id) on delete cascade,
  brand_id    uuid not null references public.brands(id) on delete cascade,
  platform     text not null,  -- instagram|twitter|facebook|pinterest|
                                -- youtube|tiktok|linkedin|medium|reddit|threads
  account_name text,
  account_id   text,
  vault_key    text,          -- Supabase Vault key for access_token
  refresh_vault_key text,     -- Supabase Vault key for refresh_token
  token_expires_at timestamptz,
  scopes       text[],
  is_active     boolean default true,
  created_at    timestamptz default now()
);

alter table public.social_accounts enable row level security;
```



```
create policy owner_only on public.social_accounts
using (owner_id = auth.uid()) with check (owner_id = auth.uid());
```

4.4 social_posts

```
create table public.social_posts (
  id                uuid primary key default gen_random_uuid(),
  owner_id          uuid not null references auth.users(id) on delete cascade,
  brand_id          uuid not null references public.brands(id) on delete cascade,
  article_id        uuid references public.articles(id) on delete set null,
  social_account_id uuid references public.social_accounts(id) on delete set null,
  platform          text not null,
  content           text not null,
  media_urls        text[] default '{}',
  hashtags          text[] default '{}',
  status            text default 'draft', -- draft|scheduled|published|failed
  scheduled_at      timestampz,
  published_at      timestampz,
  platform_post_id  text,
  error_message     text,
  created_at        timestampz default now()
);
alter table public.social_posts enable row level security;
create policy owner_only on public.social_posts
using (owner_id = auth.uid()) with check (owner_id = auth.uid());
```

4.5 analytics_cache

```
create table public.analytics_cache (
  id          uuid primary key default gen_random_uuid(),
  brand_id    uuid not null references public.brands(id) on delete cascade,
  cache_key   text not null, -- e.g. 'ga4_overview_7d'
  data        jsonb not null,
  fetched_at  timestampz default now(),
  expires_at  timestampz not null, -- fetched_at + 6 hours
  unique(brand_id, cache_key)
);
-- No RLS needed: accessed only via service role key in server-side route handlers
```

5. App Routes & Pages

Every URL, what it shows, and what it requires

Route	Page / Component	What It Does
/	Landing page (MakerKit marketing layout)	Public. Hero section with tagline, feature highlights, and Login CTA. No sign-up button — owner only.
/auth/login	Login form (MakerKit)	Email + password. Magic link option. Redirect to /home on success.
/home	Brand picker / overview	Protected. Lists all brands as cards with logo, name, and post count. 'New Brand' button triggers CreateBrandModal.
/home/brands/[brandSlug]	Brand dashboard	Protected. Overview cards: articles published, scheduled posts, traffic last 7 days. Recent articles table. Quick actions.
/home/brands/[brandSlug]/content	Content library	All articles for this brand. Filter by status (draft/ready/published). Search. Sort by date. 'New Article' button.
/home/brands/[brandSlug]/content/new	AI Content Studio	6-step wizard. Steps: Research → Outline → Draft → SEO → Social → Publish. State saved in articles table as user progresses.
/home/brands/[brandSlug]/content/[articleId]	Article editor	TipTap editor for an existing article. Auto-save on change. Publish to WordPress button.
/home/brands/[brandSlug]/social	Social calendar	FullCalendar monthly view. Posts shown as coloured events per platform. Click event to edit/reschedule.
/home/brands/[brandSlug]/social/compose	Post composer	Manual social post creation. Select platform, write content, pick schedule date, attach image.
/home/brands/[brandSlug]/analytics	Analytics dashboard	GA4 overview + top pages. GSC clicks + impressions + top queries. AdSense revenue. 7d / 30d / 90d toggle. Recharts line + bar charts.
/home/brands/[brandSlug]/settings	Brand settings	Three tabs: (1) Identity — name, tagline, logo, niche, primary color. (2) Connections — WordPress URL/username/app password, GA4 property ID, GSC property URL, AdSense ID. (3) Automation — N8N webhook URL, Postiz webhook URL, Mixpost webhook URL, extra_integrations JSON editor. Social channel URL fields (informational).

Route	Page / Component	What It Does
/api/mcp	App MCP server	Exposes per-brand config as MCP tools. N8N, Postiz, and other external tools call this to read/write brand settings without touching Supabase directly.
/api/ai/research	AI route handler	POST. Calls Perplexity Sonar Pro. Returns research brief for a given topic + brand niche.
/api/ai/outline	AI route handler	POST. Calls Gemini Flash. Returns structured article outline as JSON.
/api/ai/draft	AI route handler	POST. Calls Claude Sonnet 4.6. Returns full article draft in HTML.
/api/ai/seo	AI route handler	POST. Calls Gemma 4 31B (free). Returns meta title, meta description, alt text, internal link suggestions.
/api/ai/social	AI route handler	POST. Calls Claude Haiku. Returns platform-specific repurposed posts (one per requested platform).
/api/publish	WordPress publish handler	POST. Uses wordpress.ts client. Creates/updates wp-json post. Returns wp_post_id and url.

6. Feature Specifications

Exact behaviour for each feature — what to build, not when

6.1 Brand Management

Action	Behaviour
Create brand	Modal form: name (required), niche, tagline, logo upload. Slug auto-generated from name + 6-char UUID suffix. Slug is immutable after creation. Owner_id set to auth.uid() server-side.
Switch brand	BrandSwitcher dropdown in sidebar header. Navigates to /home/brands/[newSlug]. BrandContext re-reads from URL param on layout mount.
Brand settings	Three-tab settings page. All saves via Server Actions. WordPress app password saved to Vault on first entry; only vault_key stored in brands table. Password field shows '.....' after save. Delete brand button (with confirmation) hard-deletes brand + cascade.
Logo upload	Supabase Storage. Path: brand-assets/{owner_id}/{brand_id}/logo.png. Resize to max 400×400 on client before upload. Return public URL, store in brands.logo_url.

6.2 AI Content Studio — 6 Steps

How Steps Work

The wizard saves state at each step into the articles row (research_data, outline_data, etc. columns). If the user navigates away and returns, the wizard resumes from the last completed step. Each step calls its own /api/ai/* route handler.

Step	Name	Input	AI Call	Output
1	Research	Topic/keyword + brand niche	Perplexity Sonar Pro — web search enabled	Research brief: SERP analysis, top competing articles, content gaps, recommended angles. Saved to articles.research_data.
2	Outline	Research brief + target keyword	Gemini 2.0 Flash	H1 title (5 options), section headings (H2/H3), word count estimate per section. Drag-and-drop outline reordering via @dnd-kit. Saved to articles.outline_data.
3	Draft	Approved outline + research brief	Claude Sonnet 4.6	Full article draft in HTML rendered in TipTap editor. User can edit before proceeding. Duplicate score checked against brand's existing articles. Saved to articles.content.

Step	Name	Input	AI Call	Output
4	SEO	Draft content + focus keyword	Gemma 4 31B (free)	Meta title, meta description, focus keyword density report, internal link suggestions from brand's own articles, featured image alt text. Saved to articles.seo_data.
5	Social	Article title + excerpt + user selects platforms	Claude Haiku 4.5	One repurposed post per selected platform, formatted for that platform (hashtags for Instagram, thread structure for Twitter, professional for LinkedIn, etc.). Saved to articles.social_data.
6	Publish	Final content + SEO data	No AI — direct API call	Publish to WordPress via /api/publish (wp-json REST). Updates articles.wp_post_id, wp_post_url, status='published'. Social posts saved to social_posts table with scheduled_at set.

6.3 Social Media Calendar

Feature	Behaviour
Calendar view	@fullcalendar/react monthly view. Each social_post shown as a coloured event — one colour per platform. Click to open post detail panel.
Post scheduling	User sets scheduled_at datetime. Trigger.dev every-minute cron job queries social_posts where status='scheduled' AND scheduled_at <= now(). Calls platform API. Updates status to 'published' or 'failed' with error_message.
Post composer	Select platform (from brand's connected social_accounts). Write content. Character counter per platform (Twitter: 280, LinkedIn: 3000, etc.). Attach image from article or upload new. Pick date/time.
Auto-populated from Studio	When Step 5 (Social) completes, repurposed posts are added to social_posts with status='draft'. User reviews them in calendar view and sets scheduled_at to activate.
10 platforms	Instagram, Twitter/X, Facebook, Pinterest, YouTube (community post), TikTok (description + caption), LinkedIn, Medium (article title + teaser), Reddit (title + link), Threads.

6.4 Analytics Dashboard

Section	Data Source	Behaviour
Overview KPIs	GA4 (via MCP tool)	Sessions, pageviews, users, avg session duration. 7d / 30d / 90d selector. Recharts LineChart.

Section	Data Source	Behaviour
Top Pages	GA4	Table: page path, sessions, pageviews, avg time. Sortable.
Search Performance	GSC (via MCP tool)	Clicks, impressions, CTR, avg position. Top queries table. Recharts LineChart for trend.
Revenue	AdSense API	Estimated earnings. Page RPM. Impressions. LineChart by day.
Cache	analytics_cache table	All API responses cached for 6 hours. Server checks cache before calling API. Stale data shown with 'Last updated X hours ago' badge.

6.5 MCP Server at /api/mcp

The app exposes its own MCP server using @vercel/mcp-adapter. This lets N8N, Postiz, and other external tools call the app to read and write per-brand config without direct Supabase access.

MCP Tool Name	What It Does
get_brand_config	Given a brand slug, returns the full brand config: WordPress URL, username, GA4 property ID, GSC URL, webhook URLs, social channel URLs. WordPress app password is fetched from Vault server-side and returned only if caller provides valid owner API key in Authorization header.
list_brands	Returns all brands belonging to the authenticated owner: id, name, slug, niche, tagline, wordpress_url, n8n_webhook_url.
get_articles	Returns articles for a brand with optional status filter. Used by N8N to check what content exists before scheduling new jobs.
create_article_stub	Creates a minimal article row with title + brand_id. Used by N8N to register an article before the full Content Studio flow.

7. Supabase Vault — Secret Management

WordPress passwords and OAuth tokens NEVER stored as plain text

```
// lib/vault.ts – server-side only, uses service role key
import { createClient } from '@supabase/supabase-js';

const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

export async function storeSecret(name: string, secret: string): Promise<string> {
  const { data } = await supabaseAdmin.rpc('vault.create_secret', {
    new_secret: secret,
    new_name: name,
  });
  return data; // returns vault key UUID
}

export async function readSecret(vaultKey: string): Promise<string> {
  const { data } = await supabaseAdmin
    .from('vault.decrypted_secrets')
    .select('decrypted_secret')
    .eq('id', vaultKey)
    .single();
  return data?.decrypted_secret ?? '';
}
```

What	Where Stored	Rule
WordPress app password	Supabase Vault	Call storeSecret() in /api/brands/settings route handler on save. Store returned key in brands.wordpress_vault_key. NEVER log or return the plain password.
Social OAuth access tokens	Supabase Vault	Store per social_account. Keep vault_key in social_accounts.vault_key. Use readSecret() inside Trigger.dev job when posting — not in browser.
GA4 / GSC OAuth tokens	Supabase Vault or Google's own token store	Use google-auth-library server-side. Refresh tokens stored in Vault.
API keys (OpenRouter, Trigger.dev)	Environment variables (.env.local)	These are app-level secrets, not per-brand — env vars are correct.

8. Background Jobs — Trigger.dev

Social post scheduling and sync tasks

Job	Trigger	What It Does
publishScheduledPosts	Every 1 minute (cron)	Query social_posts WHERE status='scheduled' AND scheduled_at <= now() AND owner_id = ?. For each post: read vault_key from social_accounts, call platform API, update status to 'published' or 'failed' with error_message.
syncWordPressPosts	Manual trigger or daily cron	Call wp-json/wp/v2/posts for a brand's WordPress site. Upsert into articles table with status='published'. Used to import historical posts.
refreshAnalyticsCache	Every 6 hours per brand	Fetch GA4 + GSC overview data for each brand. Upsert into analytics_cache. Keeps dashboard fast with zero cold-fetch delay.

9. Environment Variables

Complete list for apps/web/.env.local

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
```

Server-side only. NEVER expose to browser.

```
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...
```

OpenRouter (all AI calls)

```
OPENROUTER_API_KEY=sk-or-...
```

Cloudflare AI Gateway

Format: https://gateway.ai.cloudflare.com/v1/{account_id}/brandnest/openrouter

```
CLOUDFLARE_AI_GATEWAY_URL=https://gateway.ai.cloudflare.com/v1/XXXX/brandnest/openrouter
```

Trigger.dev

```
TRIGGER_API_KEY=tr_dev...
TRIGGER_SECRET_KEY=...
```

Google APIs (server-side only)

```
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
```

Social Platform App Credentials (app-level, not per-brand)

```
INSTAGRAM_APP_ID=
INSTAGRAM_APP_SECRET=
FACEBOOK_APP_ID=
FACEBOOK_APP_SECRET=
PINTEREST_APP_ID=
PINTEREST_APP_SECRET=
LINKEDIN_CLIENT_ID=
LINKEDIN_CLIENT_SECRET=
```

Per-brand secrets (WordPress passwords, OAuth tokens) are stored
in Supabase Vault – NOT in environment variables.

10. AGENTS.md — Rules for the AI Coding Agent

Paste this into AGENTS.md at the repo root

What Is AGENTS.md

This file is read by Google Antigravity IDE (and any AI coding agent) at the start of every session. It sets the rules the AI must follow for every file it generates. Keep it at the repo root.

```
# AGENTS.md – BrandNest Coding Rules
# Read this file before writing any code.

## Architecture
- Framework: Next.js 15 App Router (TypeScript). NO Pages Router.
- Monorepo: Turborepo + pnpm. Apps live in apps/web/. Shared code in packages/.
- Database: Supabase PostgreSQL. ALL tables have owner_id and RLS.
- Auth: Supabase GoTrue (via MakerKit). Never roll custom auth.
- Background jobs: Trigger.dev only. No setTimeout, no setInterval, no API polling.
- Hosting: Vercel serverless. No NestJS, no Express, no Docker.

## User Model – CRITICAL
- ONE user type: the owner. NO teams. NO roles. NO invitations. NO public sign-up.
- NEVER create a brand_members table. NEVER add a 'role' column to any user-brand join.
- NEVER create an invitation or pending_members table.
- Single owner is enforced by Supabase RLS: USING (owner_id = auth.uid()).
- Application code ALSO filters by owner_id for defence in depth.

## Brand Routing
- Active brand = URL param: /home/brands/[brandSlug].
- BrandContext reads from URL param – NOT from React state or localStorage.
- Brand slug is auto-generated on create. NEVER allow slug changes after creation.

## Secrets – CRITICAL
- WordPress app passwords go in Supabase Vault. NEVER in the brands table as plain text.
- OAuth tokens go in Supabase Vault. NEVER in social_accounts as plain text.
- Use lib/vault.ts for all Vault reads/writes. Vault access requires service role key.
- The service role key (SUPABASE_SERVICE_ROLE_KEY) is NEVER passed to the browser.
- Browser client uses NEXT_PUBLIC_SUPABASE_ANON_KEY only.

## AI Calls
- ALL AI calls go through lib/ai-router.ts → Cloudflare AI Gateway → OpenRouter.
- Use the cheapest model that works. Free Gemma 4 31B for titles/meta/hashtags.
- NO direct fetch() to api.openai.com or api.anthropic.com – always via gateway.

## File Conventions
- Server Actions: apps/web/app/home/**/_actions/*.ts
- Route Handlers: apps/web/app/api/**/*.*ts
- DB migrations: apps/web/supabase/migrations/YYYYMMDD_description.sql
- Never add a useEffect to fetch data – use Server Components with async/await.
- Never store auth tokens in localStorage or cookies manually – Supabase handles this.

## What NOT to build
- Do NOT build: team management, billing/Stripe, public sign-up, multi-tenant isolation, custom auth, Pages Router, REST APIs outside /api/, Docker containers.
- Do NOT use: Prisma (use Supabase client), Redux (use React context + server state), class components, styled-components.
```

11. Reference Repositories

Give these to the AI as context — do not copy-paste code blindly

Repository	What to Learn From It
github.com/makerkit/next-supabase-saas-kit-lite	Base structure. Auth flows. Sidebar layout. Server Actions pattern. MakerKit conventions for protected routes.
github.com/maazajaz/socialens	Social calendar implementation. Post composer pattern. Platform connection flows. FullCalendar + Supabase integration.
github.com/drylikov/CMS	TipTap rich text editor wired to Next.js App Router. Auto-save pattern. Content serialisation to HTML/JSON.
github.com/TailAdmin/free-nextjs-admin-dashboard	Dashboard UI patterns. Sidebar nav. Stat cards. Responsive admin layout with shadcn/ui.
github.com/run-llama/mcp-nextjs	How to build and expose an MCP server from a Next.js route handler.
github.com/physics91/openrouter-mcp	OpenRouter MCP integration. Multi-model routing. How to register GA4, GSC, WordPress as tools.
github.com/wheattoast11/openrouter-deep-research-mcp	Deep research MCP setup. Perplexity Sonar Pro web search integration.

12. Non-Functional Requirements

Performance, security, and UX constraints

Requirement	Target	Implementation
TTFB	< 500ms	Next.js Server Components. Supabase connection pooling. Vercel Edge CDN.
Analytics load time	< 300ms for cached data	analytics_cache table with 6-hour TTL. Serve stale while revalidating.
Dark mode	Native — no flash	MakerKit + shadcn/ui CSS variables. system preference respected. Theme toggle in sidebar.
Data isolation	Zero cross-brand leakage	RLS on every table. Application code always filters by owner_id. Server Actions verify auth before every DB call.
Slug immutability	Slugs never change	Slug set by DB trigger on INSERT. No UPDATE allowed on slug column via application code. 301 redirect logic not needed.
Mobile responsive	Usable on tablet/mobile	TailwindCSS responsive prefixes. Sidebar collapses to hamburger on sm screens.
Empty states	Every list has one	ContentLibrary, SocialCalendar, and AnalyticsDashboard each have a purposeful empty state with a clear call to action when data is absent.
Error boundaries	Every route	Next.js error.tsx per route segment. Toast notifications for failed AI calls.

13. MVP Acceptance Criteria

You are done with MVP when every item below is true

MVP Scope Definition

MVP = brand configuration hub. The app is useful the moment external tools (N8N, Postiz) can read per-brand credentials from /api/mcp. The AI Content Studio, social calendar, and analytics are full product features.

#	Acceptance Criterion
1	Owner can log in with email/password and is redirected to /home
2	Owner can create a brand with name, niche, tagline, and logo — slug auto-generated and shown in URL
3	Owner can create 5+ brands and switch between them instantly using BrandSwitcher — each brand opens at /home/brands/[brandSlug]
4	Brand settings page saves WordPress URL, username, and app password — password stored in Supabase Vault, not visible in plain text after save
5	Brand settings saves GA4 property ID, GSC property URL, AdSense publisher ID
6	Brand settings saves N8N webhook URL, Postiz webhook URL, Mixpost webhook URL, and extra_integrations JSON
7	Brand settings saves all 10 social channel profile URLs (Instagram through Threads)
8	/api/mcp returns correct per-brand config for a given brand slug — WordPress URL, credentials (from Vault), webhook URLs, social URLs
9	N8N can call /api/mcp and receive complete config for any brand by slug without direct Supabase access
10	Deleting a brand hard-deletes it and all its articles, social accounts, posts, and analytics cache — no orphaned rows

Full Product Acceptance Criteria (Post-MVP)

#	Acceptance Criterion
11	AI Content Studio Step 1 (Research) returns a research brief via Perplexity Sonar Pro for any given topic and brand niche
12	Step 2 (Outline) produces a structured H1/H2/H3 outline — user can drag-and-drop to reorder sections
13	Step 3 (Draft) generates a full article in TipTap editor — user can edit before proceeding
14	Step 4 (SEO) returns meta title, meta description (via Gemma 4 31B free), and internal link suggestions from the brand's own article library
15	Step 5 (Social) generates platform-specific repurposed posts for selected platforms
16	Step 6 (Publish) pushes article to WordPress via wp-json REST and records wp_post_id in the articles table

#	Acceptance Criterion
17	Social calendar shows all scheduled posts. Trigger.dev job publishes them at the correct time.
18	Analytics dashboard displays GA4 + GSC data with 6-hour cache — loads in < 300ms when cache is warm
19	Duplicate score is calculated and shown in Step 3 before the user proceeds to SEO
20	All features work in dark mode with no visual defects

14. Setup Commands

Run these once to initialise the project

```
# 1. Clone MakerKit Lite (replace with your fork URL if you have one)
git clone https://github.com/makerkit/next-supabase-saas-kit-lite brandnest
cd brandnest

# 2. Install all dependencies
pnpm install

# 3. Copy env example and fill in your keys
cp apps/web/.env.local.example apps/web/.env.local
# Edit apps/web/.env.local with your Supabase, OpenRouter, Cloudflare keys

# 4. Start Supabase locally
cd apps/web && npx supabase start

# 5. Apply all migrations
pnpm run supabase:web:reset

# 6. Run the dev server
pnpm run dev

# 7. Connect git remote
git remote set-url origin https://github.com/YOUR-USERNAME/brandnest
git push -u origin main

# 8. Connect Vercel (auto-deploys on every push to main)
npx vercel link
npx vercel env pull apps/web/.env.local
```

What to Remove from MakerKit Lite

- Remove or disable any team invite / member management UI in MakerKit (search for 'invitation' and 'team member' in the codebase)
- Remove Stripe / billing pages — this is a personal tool, not a SaaS
- Disable public sign-up in Supabase: Dashboard → Auth → Providers → Email → uncheck 'Enable Sign Ups'
- Remove the Pricing page from the MakerKit marketing layout

How to Use This Document

Open AGENTS.md in your IDE root and paste Section 10. Then feed this entire document to your AI coding assistant as the project context. Start with the MVP scope (Sections 1, 3, 4, 5 settings route, 7, 8) and work toward the full product once /api/mcp is working and N8N can read brand config.